

Modules

Introduction — Cosa sono i moduli e perché esistono

Definizione. Un **modulo** è un raggruppamento di **uno o più pacchetti** (package) correlati, descritto da un file speciale chiamato `module-info.java`. È un livello di incapsulamento *superiore* al package: mentre `public/protected/private` controllano la visibilità a livello di classe e package, il modulo controlla la visibilità a livello di *interi package verso l'esterno*.

I moduli servono a definire **confini** (boundaries) e **dipendenze** ben precise all'interno del codice. Come sviluppatore dichiarare due cose: quali package del tuo modulo sono accessibili dall'esterno, e da quali altri moduli il tuo modulo dipende.

Perché sono stati introdotti (i 4 vantaggi principali):

1. **Controllo degli accessi più forte.** Prima avevi solo i modificatori di visibilità classici. Ora puoi rendere un package visibile *solo* a determinati moduli. Questo migliora il disaccoppiamento e ti permette di nascondere i dettagli implementativi.
2. **Riduzione della dimensione dell'applicazione.** Java come piattaforma è enorme. Con la modularizzazione puoi dichiarare *solo i moduli delle API Java che ti servono* davvero, ottenendo un'applicazione più piccola. Utile per dispositivi con memoria limitata (es. telefoni).
3. **Rilevamento precoce del codice mancante.** Prima dei moduli, una classe o un `.jar` mancante veniva scoperto *a runtime*, nel momento in cui quella classe veniva usata per la prima volta. Con i moduli, la JVM controlla l'**intero grafo delle dipendenze all'avvio**: se manca un modulo lo scopri subito, non ore dopo quando l'utente clicca sul bottone sbagliato.
4. **Miglior disaccoppiamento** e struttura più chiara su larga scala.

Concetto chiave sull'incapsulamento forte: per impostazione predefinita, **tutte le classi di tutti i package di un modulo sono fortemente incapsulate**. Questo significa che nessun altro modulo può usarle, *anche se sono classi public*. Se vuoi che una classe sia visibile fuori dal modulo, devi esportare esplicitamente il suo package.

Namespace separati: i nomi dei package e i nomi dei moduli vivono in spazi dei nomi diversi. Puoi quindi avere un modulo che si chiama `farm.stock.account` e un package che si chiama esattamente `farm.stock.account` senza alcun conflitto. Negli esempi del corso questo capita spesso.

Dov'è il `module-info.java`: va posizionato nella **radice dell'albero dei sorgenti** del modulo. Non devi elencare i package che contiene: vengono inclusi automaticamente in virtù della loro posizione sotto quella radice.

L'esempio "farm" (fattoria) che ricorre in tutto il corso: c'è un'applicazione aziendale con più moduli — `farm.animals`, `farm.owner`, `farm.vet`, moduli per la contabilità e le vendite — collegati da frecce che rappresentano le dipendenze. Es: il modulo veterinario dipende dal proprietario, il proprietario dipende dagli animali.

Compiler and Runtime flags — Compilare ed eseguire dalla riga di comando

Questa lezione spiega i **flag** di `javac` (compilatore) e `java` (runtime/JVM) da usare in PowerShell o prompt dei comandi.

Compilazione con `javac`:

- `-d <directory>` → dice al compilatore **dove mettere i file `.class`** prodotti (tipicamente una cartella `out`).
- Poi elenchi tutti i file sorgente, incluso il `module-info.java`.

Esempio "manuale" (elencando i file):

```
javac -d out src/com/firm/.../*.java src/module-info.java
```

Usando le "**module directories**" (più comodo, così non elenchi ogni singolo file):

- `--module-source-path <cartella>` → dove si trova il codice sorgente dei moduli.
- `--module <nome>` (versione breve: `-m`) → il nome del modulo da compilare. Puoi separarne più di uno con la virgola.

Regola importante: se usi una module directory, il **nome della cartella deve corrispondere esattamente al nome del modulo** dichiarato nel `module-info.java`, altrimenti non funziona.

Esecuzione con `java` (la JVM):

- `--module-path <cartella>` (versione breve: `-p`) → è il "class path" equivalente ma per le applicazioni modulari; indica dove stanno i moduli/artefatti. Spesso la cartella si chiama `mods`.
- `--module <modulo>/<classe>` (versione breve: `-m`) → specifica il nome del modulo, uno **slash**, e poi il **nome completamente qualificato della classe main** (package + classe).

Esempio:

```
java -p out -m test.module.firm/com.firm.Main
```

Riassunto dei flag (come li descrive il docente):

Fase	Flag	Significato
<code>javac</code>	<code>-d</code>	dove mettere i <code>.class</code>
<code>javac</code>	<code>-m / --module</code>	nome del modulo da compilare
<code>javac</code>	<code>--module-source-path</code>	dove sta il codice sorgente
<code>javac/java</code>	<code>-p / --module-path</code>	dove stanno i moduli dipendenti
<code>java</code>	<code>-m / --module</code>	modulo/classe-main da eseguire (nome pienamente qualificato)

Hello World Example — Il primo modulo funzionante

Qui il docente mostra in pratica (usando l'editor Atom) un progetto chiamato `test.module.firm`.

Il `module-info.java` più semplice possibile:

```
module test.module.firm {  
  
}
```

Vuoto. Non esporta nulla. Conseguenza: **tutto ciò che c'è dentro è fortemente incapsulato**, nessun altro modulo può accedervi. La classe `Main` resta comunque eseguibile perché la esegui *dall'interno* del modulo stesso.

Due modi mostrati:

1. **Elencando tutti i file** (senza module directory): fai `javac -d out` seguito da tutti i path dei `.java`, poi esegui con `java -p out -m test.module.firm/com.firm.Main` → output: `Hello module world`.
2. **Con la module directory** (secondo progetto): crei una cartella il cui nome **coincide** col nome del modulo, ci metti dentro `module-info.java`, e compili con `--module + --module-source-path`. Il compilatore trova la cartella, legge il `module-info.java`, verifica che il nome dichiarato (riga 1) corrisponda, e compila. L'esecuzione è identica.

La lezione insegna *dove va fisicamente* il `module-info.java` e come cambiano i comandi tra i due approcci.

Dependencies — requires ed exports

Un singolo modulo isolato serve a poco. I moduli collaborano, e qui entrano le due parole chiave fondamentali.

Definizioni:

- **exports <package>** → rende un **package** visibile all'esterno del modulo. Si esporta *un package*.
- **requires <modulo>** → dichiara che il tuo modulo **dipende** da un altro **modulo** (ne usa i tipi/codice). Si richiede *un modulo*.

Trucco mnemonico del docente: *exports* ha la **P** (di *Package*) a destra; *requires* riguarda un **Modulo**. In sintesi: **si esporta un package, si richiede un modulo**.

Dipendenza implicita da `java.base`: *tutti* i moduli dipendono implicitamente dal modulo base di Java. Ecco perché quando usi `String` o `Object` (che stanno in `java.lang`) funziona senza dichiarare nulla: `java.lang` è uno dei package esportati da `java.base`.

Incapsulamento selettivo: solo i package che elenchi con `exports` diventano visibili. **Tutti gli altri restano fortemente incapsulati**. Questo è il punto centrale.

Esempio (le due facce di una dipendenza):

Modulo *produttore* — `farm.stock.account`:

```
module farm.stock.account {  
    exports farm.stock.account; // espone il package  
}
```

La classe `HowMany` dentro quel package ha un metodo pubblico `getCount()` che restituisce, es., `11`.

Modulo *consumatore* — `farm.stock.sale`:

```
module farm.stock.sale {  
    requires farm.stock.account; // dipende dall'altro modulo  
}
```

La classe `Sell` importa `HowMany`, ne crea un'istanza e chiama `getCount()`.

Compilazione con più moduli: `-d out` per l'output, `--module`

`farm.stock.account`, `farm.stock.sale` per i due moduli, `--module-source-path src`.

Esecuzione: `java -p out -m farm.stock.sale/farm.stock.sale.Sell` — nota la struttura `nomeModulo/nomePackage.NomeClasse`.

La dimostrazione dell'errore: se rimuovi `exports farm.stock.account` dal modulo produttore e riesegui, ottieni un **errore di accesso**: la classe `HowMany` non è più accessibile perché il suo package non è più esportato. Ottima prova concreta di cosa fa (e non fa) `exports`.

Impacchettare in un .jar: con lo strumento `jar`:

```
jar -c -v -f mods/farm.stock.sale.jar -C out/farm.stock.sale .
```

- `c` = create, `v` = verbose, `f` = file (nome del jar).
- `-C out/farm.stock.sale .` = "vai in questa cartella e includi tutto quello che c'è sotto (il `.`)".
- La cartella `mods` deve già esistere ed è il nome convenzionale per gli artefatti.

Poi esegui puntando `-p` alla cartella `mods` invece che a `out`: ora giri il codice **dal jar**.

Opens and requires transitive — Reflection e dipendenze transitive

Due argomenti distinti in una lezione.

opens — accesso via reflection

Il problema. Framework come **Spring** e **Hibernate** hanno bisogno di accedere al tuo codice *a runtime* tramite **reflection** (una tecnica in cui, dato il nome di un package e di una classe come stringhe, si costruisce/ispeziona l'oggetto dinamicamente). La reflection può *bypassare i normali controlli di accesso*, quindi con l'incapsulamento forte dei moduli va autorizzata esplicitamente.

La soluzione — opens. Vuoi il meglio dei due mondi: **incapsulamento forte in fase di compilazione**, ma **accesso via reflection a runtime**. La keyword `opens` fa esattamente questo.

Sintassi:

```
opens <package>;           // apre a chiunque, via reflection
opens <package> to <modulo>; // apre solo a un modulo specifico
```

Puoi anche aprire l'intero modulo:

```
open module farm.stock.sale {
    ...
}
```

Un `open module` apre automaticamente *tutti* i suoi package alla reflection, quindi le singole righe `opens` diventano superflue.

Differenza chiave: `exports` dà accesso *in compilazione* (puoi scrivere codice contro quei tipi); `opens` dà accesso *a runtime via reflection*. Con `opens . . .` puoi compilare contro i tipi ma è la reflection a poterli raggiungere a runtime.

requires transitive — dipendenze che si propagano

Il problema (con l'esempio farm). Abbiamo `farm.vet` → `farm.owner` → `farm.animals`. Domande:

1. Perché `farm.vet` dovrebbe aver bisogno di dichiarare a sua volta `requires farm.animals`, se già dipende da `farm.owner` che dipende da `farm.animals`?
2. La classe `Owner` (nel modulo `farm.owner`) ha un metodo `getAnimal()` che **restituisce un tipo `Animal`** — ma `Animal` *non* sta in `farm.owner`, sta in `farm.animals`!

Quindi parte dell'**API pubblica** di `Owner` espone un tipo che appartiene a un *altro* modulo. Chiunque usi `owner.getAnimal()` ha bisogno del tipo `Animal`, e quindi ha bisogno del modulo `farm.animals`.

Cos'è la transitività (spiegata con analogie): "più alto di" è transitiva — se A è più alto di B e B più alto di C, allora A è più alto di C. "Vive accanto a" *non* è transitiva. La dipendenza transitiva funziona come "più alto di".

La soluzione — `requires transitive`:

```
module farm.owner {
    exports farm.owner;
    requires transitive farm.animals; // riga chiave
}
```

Con `requires transitive farm.animals`, **qualsiasi modulo che richiede `farm.owner` ottiene automaticamente anche `farm.animals`**. Così `farm.vet`, che fa

`requires farm.owner`, ottiene la dipendenza da `farm.animals` *implicitamente*, senza doverla dichiarare, e senza rischiare doppie dichiarazioni conflittuali. Risolve entrambi i problemi di sopra in un colpo solo.

Il codice dell'esempio: `Cow` estende `Animal`; `Cattle` contiene una lista di mucche con tag `C1`, `C2`, `C3`; `Owner.getAnimal(tag)` scorre la lista e restituisce l'animale con quel tag (usando *copia difensiva* quando ritorna la lista). Il `Main` del veterinario chiama `owner.getAnimal("C2")` → stampa il tag `C2`, e mostra che ci sono 3 bovini. Funziona perfettamente grazie al `requires transitive`.

Operations on a Module — `jar`, `java --describe-module`, `jdeps`, `jmod`

Questa lezione elenca le **operazioni** che puoi fare su un modulo (questa sezione compare due volte nella trascrizione, è ripetuta identica).

Gli strumenti:

- **jar** — crea un archivio (una specie di "zip eseguibile" trasportabile). È il modo standard per impacchettare i moduli.
- **java --describe-module / jar --describe-module** — per **descrivere/ispezionare** un modulo.
- **jdeps** — Java **dependency analysis tool**: scopre le dipendenze reali di un modulo.
- **jmod** — per lavorare con i file **JMOD**; pensato per **librerie native** e artefatti che *non* possono stare in un `.jar`. Oracle consiglia di usare i normali `.jar` per i file di classe. I **JMOD non sono eseguibili**: servono con lo strumento **jlink** per creare un'immagine nativa personalizzata. Tutti i moduli del JDK stanno in formato JMOD sotto `JAVA_HOME/jmods`.

Impacchettare i moduli farm in jar:

```
jar -c -v -f mods/farm.vet.jar -C out/farm.vet .  
jar -cvf mods/farm.owner.jar -C out/farm.owner .  
jar -cvf mods/farm.animals.jar -C out/farm.animals .
```

(prima con flag lunghi, poi versione breve `cvf`). Il `module-info.class` finisce dentro il jar rendendolo un **jar modulare**. Poi esegui puntando `-p` a `mods`: funziona come prima ma con jar eseguibili.

Descrivere un modulo dentro un jar (senza aprirlo a mano):

```
java --describe-module --module-path mods --module farm.animals
```

Ti dice cosa **esporta** (es. i package del bestiame) e cosa **richiede** (sempre `java.base`, obbligatorio in ogni modulo). Versione breve `-d`. Funziona anche con `jar --describe-module -f mods/farm.animals.jar`.

jdeps — differenza rispetto a **--describe-module**: **--describe-module** legge solo ciò che è *dichiarato* nel `module-info`. **jdeps** guarda anche il **codice** e ti dice le dipendenze *effettivamente presenti*.

```
jdeps -s --module-path mods mods/farm.animals.jar
```

- `-s` (o `--summary`) = riepilogo.
- `--module-path` serve per trovare i moduli dipendenti.

Risultato: `farm.animals` → `java.base`; `farm.owner` → `farm.animals` + `java.base`; `farm.vet` → `farm.owner` + **`farm.animals`** + `java.base` (l'ultima grazie al `requires transitive` di `farm.owner`).

jmod describe: ispezionando i JMOD del JDK (es. `java.se`) vedi tutti i moduli richiesti, tutti transitivi — utile per scegliere i moduli minimi per un'immagine nativa personalizzata con `jlink`.

Services — Disaccoppiare i moduli con **provides** / **uses**

Questa è concettualmente la lezione più importante e ricca.

Il problema. Finora i moduli sono **strettamente accoppiati** (`requires` + `exports`): il consumatore deve conoscere il tipo concreto del produttore. Se hai un veterinario che dipende da tante implementazioni diverse di "proprietario" (`farm.owner`, `farm.neighbour`, ecc.), e magari anche un ispettore che dipende dalle stesse implementazioni, l'accoppiamento esplode.

La soluzione — i Services. Si introduce un **registro** (service registry, middleware) che astrae i **consumatori** dai **produttori**. Il legame tra i due è un **contratto di servizio**, cioè un'interfaccia, ospitata nel registro. Il consumatore chiede al registro le implementazioni dell'interfaccia; **non sa da dove arrivino**.

I tre ruoli:

1. **Service contract (API)** — un modulo che contiene l'**interfaccia** e la **esporta**. Es. modulo `farm.owner.api` che esporta il package `farm.owner.api` con l'interfaccia `OwnerService`.
2. **Consumer (consumatore)** — es. il modulo `farm.vet`. Nel suo `module-info` scrive:

```
module farm.vet {  
    requires farm.owner.api;    // conosce solo il contratto  
    uses farm.owner.api.OwnerService; // dice al registro: mi interessano le implementazioni di  
    // questo tipo  
}
```

La keyword **uses** **<interfaccia>** indica al registro che questo consumatore vuole le implementazioni di quel servizio. Il consumatore **non ha più requires verso i moduli concreti** dei proprietari e non sa nemmeno che esistano.

Nel codice, il consumatore usa **ServiceLoader** (da `java.util`):

```
ServiceLoader<OwnerService> loader = ServiceLoader.load(OwnerService.class);  
  
for (OwnerService s : loader) {  
    s.getAnimal("C2");  
}
```

Passi il *tipo dell'interfaccia*, il registro ti restituisce tutte le implementazioni disponibili, e tu ci iteri sopra.

3. **Provider (fornitore)** — es. modulo `farm.owner`. Nel suo `module-info`:

```
module farm.owner {  
    requires farm.owner.api;  
    provides farm.owner.api.OwnerService  
        with farm.owner.FarmOwnerService; // interfaccia ... with ... tipo concreto  
}
```

La keyword **provides <interfaccia> with <classe-concreta>** registra l'implementazione presso il registro. **Nota cruciale:** il package `farm.owner` (che contiene la classe concreta) **NON viene esportato**. Quindi il tipo `FarmOwnerService` è **incapsulato**: il consumatore non lo vedrà mai. Conosce solo l'interfaccia. Questo è il grande vantaggio del disaccoppiamento.

Un secondo provider (`farm.neighbour`) è identico ma con `with NeighbourOwnerService`, che implementa la stessa interfaccia e restituisce, es., il vicino.

Risultato all'esecuzione: il `Main` del veterinario ottiene *entrambe* le implementazioni (proprietario e vicino) dai due provider diversi, senza conoscere i loro tipi concreti. I moduli sono completamente disaccoppiati.

Riassunto delle 3 keyword dei services:

- `exports` (nel modulo API) → pubblica l'interfaccia.
- `uses <interfaccia>` (nel consumatore) → "voglio implementazioni di questo servizio".
- `provides <interfaccia> with <classe>` (nel provider) → "registro questa implementazione".
- `ServiceLoader.load(...)` (nel codice del consumatore) → recupera le implementazioni a runtime.

Services – SoftDrink Example — Esempio costruito da zero + ServiceLoader avanzato

Stesso pattern dei services, ma costruito da zero e con un dettaglio in più sul `ServiceLoader`. Il dominio è una **bibita** (soft drink) di diverse taglie.

Struttura:

- **API** — modulo `softdrink.api` con l'interfaccia `SoftDrinkService` (metodo `getSize()`). Il suo `module-info` fa `exports softdrink.api`. Espone *solo l'interfaccia*; i consumatori non conosceranno mai i tipi concreti dei provider.
- **Consumer** — modulo `softdrink.customer` (il cliente). Fa `requires softdrink.api + uses softdrink.api.SoftDrinkService`.
- **Provider** — quattro moduli: `softdrink.small`, `softdrink.medium`, `softdrink.large`, `softdrink.extralarge`. Ognuno fa `requires softdrink.api` e `provides SoftDrinkService with <classe>`, e **non esporta** il proprio package (implementazione incapsulata).

Ogni classe concreta (es. `SmallSoftDrinkService`) implementa l'interfaccia e fa override di `getSize()` restituendo la propria taglia.

Nota sulla compilazione: il modulo API compare nei `requires` di tutti gli altri, quindi viene **incluso automaticamente** — non serve elencarlo esplicitamente in `--module`; basta elencare `small/medium/large/extralarge`.

La dimostrazione "potente": `small`, `medium` e `large` vengono eseguiti dalla cartella `out`, ma **extra large viene impacchettato in un .jar** e messo in `mods`. Poi al runtime si punta `-p` a **entrambe** le cartelle, separate da `;`:

```
java -p "out;mods" -m softdrink.customer/softdrink.customer.Main
```

E magicamente nell'output compare anche `extra large`, prelevato dal jar a runtime. Questo mostra la **flessibilità estrema** dei services: puoi aggiungere implementazioni semplicemente lasciando cadere un jar nel `module-path`, senza toccare il consumatore.

ServiceLoader — i due metodi (parte finale, molto importante):

1. Iterazione (**load + for-each**):

```
ServiceLoader<SoftDrinkService> loader = ServiceLoader.load(SoftDrinkService.class);  
  
for (SoftDrinkService s : loader) { ... }
```

Problema: questo modello di iterazione **istanzia TUTTE le implementazioni** dei provider. È detto **eager** (avido). Se hai vincoli di memoria è uno spreco.

2. Stream (valutazione pigra / lazy):

```
loader.stream()  
    .filter(p -> p.type() == SmallSoftDrinkService.class) // controllo sul tipo, SENZA istanziare  
    .map(ServiceLoader.Provider::get)                       // <-- qui avviene l'istanziamento  
    .forEach(s -> s.getSize());
```

Con `stream()`, al momento della creazione dello stream **nessuna implementazione è ancora istanziata**. Puoi **filtrare** (es. tieni solo la `small` perché è "la più salutare") controllando il tipo del provider, e solo quando chiami `.get()` (nel `map`) quella *specifica* implementazione viene

istanziata. Risultato: **viene istanziato un solo provider**, la `small`. Questo è il vantaggio della valutazione lazy per risparmiare memoria.

Types of Modules — I tre tipi di modulo

Definizione: esistono tre tipi di modulo.

1. **Named modules (moduli con nome / "normali")**. Sono quelli visti finora: hanno un `module-info.java`, vivono nel **module-path** (`-p`), e li usi col flag del `module-path` invece del `class-path`.
2. **Unnamed modules (moduli senza nome)**. Sono l'esatto opposto: **jar NON modulari** (senza `module-info`; se ce n'è uno viene ignorato). Vivono nel **class-path**, il vecchio modo pre-Java-9. **Regola direzionale importante:** il codice sul **class-path** può leggere il codice sul `module-path`, **ma non viceversa** (un modulo con nome non può leggere l'unnamed module dal `class-path`).
3. **Automatic modules (moduli automatici)**. Il terzo tipo: sono jar **NON modulari** (tipicamente **librerie di terze parti**) che però metti sul **module-path**. Quando un'applicazione modulare incontra un jar senza `module-info` sul `module-path`, **Java gli assegna automaticamente un nome** e lo tratta come modulo. Tutti i suoi package vengono **esportati automaticamente** (serve dare accesso, dato che sei un modulo che deve leggerlo).

Come Java sceglie il nome di un automatic module:

1. Se nel file **MANIFEST** del jar c'è la proprietà `Automatic-Module-Name`, si usa quella.
2. Altrimenti, si applicano queste **regole di denominazione** (esempio: file `text-utils-1.0.jar`):
 - Si toglie l'estensione `.jar` → `text-utils-1.0`.
 - Si elimina l'informazione di **versione** finale (`-1.0`) → `text-utils`.
 - Si sostituiscono i caratteri non-alfanumerici con **punti** → `text.utils`.
 - Sequenze di punti multipli → un solo punto; un punto iniziale/finale viene rimosso.

Perché le regole contano: l'applicazione modulare che usa la libreria deve dichiarare `requires <nome-automatico>` nel proprio `module-info`. Quindi devi sapere che nome Java darà al jar per poterlo richiedere correttamente.

Le due demo:

- **Unnamed module:** compili una classe, la impacchetti in un jar (es. `sub.jar` in `mods`), e la esegui col **class-path** (`-cp mods/sub.jar`, con `test.Main`). Funziona sia dalla cartella `out` sia dal jar. Il punto: nel secondo caso usi il **class-path** per riferirti a un jar **senza nome di modulo** (jar non modulare, il vecchio modo, ancora disponibile per retrocompatibilità — molto importante per gli ingegneri Java).
- **Automatic module:** hai una libreria non modulare `text-utils` (una classe `Utils` che prende una stringa e la restituisce in maiuscolo). La impacchetti in un jar e la metti nella cartella `mods` dell'applicazione modulare. L'applicazione (`typing.sample`) nel suo `module-info` fa `requires text.utils` (nome generato automaticamente dal jar `text-utils`). Compili ed esegui col `module-path`. Il codice chiama

`Utils.toUpper("...")` → output in maiuscolo. Nota chiave: il jar **non è modulare** e non è elencato esplicitamente da nessuna parte, ma poiché il `module-info` dell'app fa `requires text.utils`, **Java crea automaticamente** il modulo `text.utils` dal jar. Da qui il nome "moduli automatici".

Migration Strategies — Migrare applicazioni class-path a moduli

L'ultima lezione: come **migrare** un'applicazione legacy (basata su class-path) a un'applicazione modulare. Ci sono **due strategie**.

Strategia Bottom-Up (dal basso verso l'alto)

Quando usarla: quando **controlli tutti i jar** (nessuna dipendenza esterna fuori dal tuo controllo).

Algoritmo:

1. Parti dai **nodi foglia** (leaf nodes): i moduli che **non hanno dipendenze** verso altri moduli, tranne quelli del JDK.
2. Per ciascuno, scrivi il `module-info.java` (trasformandolo in named module): assicurati di **esportare** (exports) tutti i package richiesti dai jar di livello superiore, e di dichiarare i **requires** verso i moduli da cui dipende.
3. Sposta il nuovo modulo **dal class-path al module-path**.
4. Ripeti risalendo di livello, fino a modularizzare il jar di livello più alto. A quel punto **hai finito**.

Aiuto da jdeps: puoi usare `jdeps` con il flag `--generate-module-info <dir>` per **generare automaticamente** il `module-info.java` invece di analizzare tutto il codice a mano.

```
jdeps -s --generate-module-info <dir> <jar>
```

Attenzione: la generazione automatica **esporta TUTTI i package** del jar — spesso *non* è quello che vuoi, quindi dovrai poi modificare a mano il `module-info` per esportare solo il necessario. (Nota: un jar "senza nome"/unnamed viene identificato con `U.N.` come identificatore.)

Il problema del bottom-up: se ci sono **jar esterni** che *non controlli* e che non sono modularizzati, sei bloccato. Non puoi scrivere tu il loro `module-info`, e non puoi aspettare indefinitamente che gli sviluppatori esterni migrino. **Sei vincolato all'anello più debole** della catena delle dipendenze: perdi il controllo sui tempi della migrazione.

Strategia Top-Down (dall'alto verso il basso)

Quando usarla: proprio quando hai **dipendenze esterne fuori dal tuo controllo**. È l'esatto opposto: parti dal **modulo di livello più alto** (il "tetto") e scendi.

Come gestisce i jar esterni non controllati: usando gli **automatic modules**. Sposti semplicemente il jar esterno non modulare **dal class-path al module-path**, *senza modificarlo*, e lì diventa automaticamente un automatic module. Tutto quello che devi sapere è il **nome che Java gli darà** (regole della lezione 9), così puoi scrivere il giusto `requires` nelle tue dipendenze.

L'esempio:

- Un'applicazione `app` che dipende da una `external-utils` (esterna, mantenuta da altri), la quale a sua volta dipende da un'altra `external-utils2`.
- Inizialmente tutto è sul `class-path` e funziona (immagina un'app scritta prima di Java 9).
- Parti dall'alto: modularizzi **a mano** l'applicazione (`app`), scrivendo il suo `module-info.java`, e la sposti sul `module-path`. Il modulo si chiama, es., `app.application`.
- **Problema:** un modulo sul `module-path` **non può accedere al class-path**. Ma va bene: modularizzi il jar esterno `external-utils` trasformandolo in **automatic module** (basta spostarlo sul `module-path` anche se è gestito da altri). È proprio l'`automatic module` che rende possibile la migrazione top-down senza dover aspettare i manutentori esterni.

Sintesi finale — le keyword del module system in un colpo d'occhio

Keyword	Dove	Cosa fa
<code>module X { }</code>	<code>module-info.java</code>	dichiara un modulo con nome
<code>exports <package></code>	produttore	rende un package visibile in compilazione
<code>exports <package> to <mod></code>	produttore	esporta solo verso moduli specifici
<code>requires <modulo></code>	consumatore	dichiara una dipendenza da un modulo
<code>requires transitive <modulo></code>	produttore	propaga la dipendenza a chi richiede <i>me</i>
<code>opens <package></code>	modulo	permette l'accesso via reflection a runtime
<code>open module X { }</code>	modulo	apre tutti i package alla reflection
<code>uses <interfaccia></code>	consumatore (service)	"voglio le implementazioni di questo servizio"
<code>provides <interf> with <classe></code>	provider (service)	registra un'implementazione nel registro

E i **tre tipi di modulo**: *named* (con `module-info`, sul `module-path`), *unnamed* (jar non modulari, sul `class-path`), *automatic* (jar non modulari sul `module-path`, nome generato da Java).